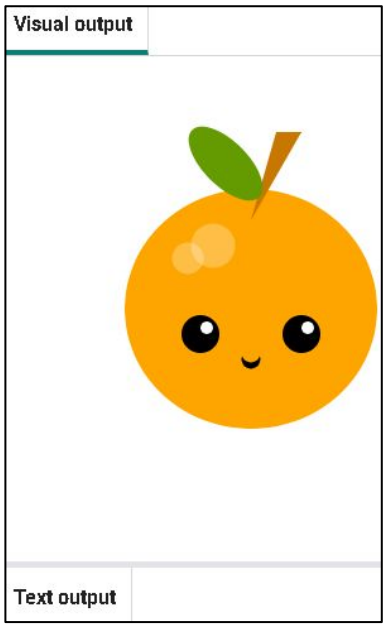
Make a face

Introduction

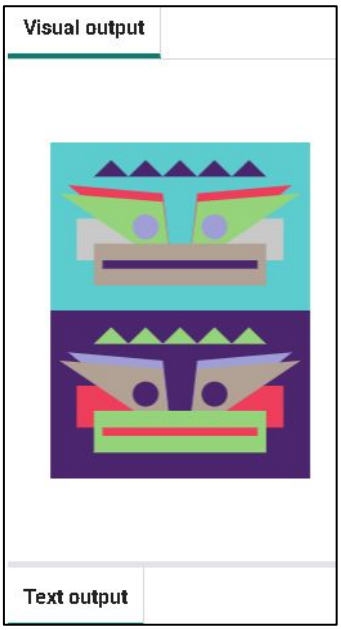
This (Design) project guides you to use your new skills and encourages you to make design choices based on your interests.
In this project, you will write Python code to draw a face or mask using geometric shapes.

Let us have a look at few of the visual outputs shown below.

Kawaii fruit



Stacked faces



Tribal mask

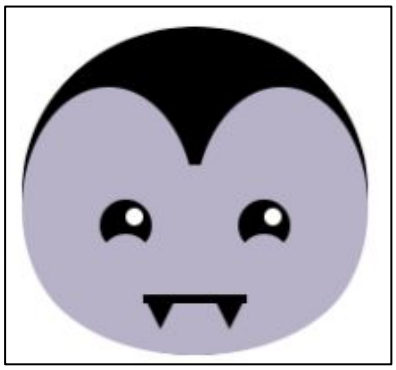
Open the following URL into your browser to explore the individual projects:

- **Kawaii fruit:** (rpf.io/fruit-face)
- **Stacked faces:** (rpf.io/stacked-faces)
- **Tribal mask:** (rpf.io/tribal-mask)

Choose a background colour

This is the first step where you decide the kind of face or mask you want to create. Use it to plan your design and prepare your canvas.

Type the URL (rpf.io/mf-starter) in your browser to open the starter project



The three numbers in `background(0, 0, 0)` are red, green and blue values. Experiment with changing the numbers to any whole number between 0 and 255 to change the background colour.

```
def draw():
    # Put code to run every frame here
    background(0, 0, 0)
```

Test: Run your code and you should see a coloured square as your background.

Circles and ovals

Draw and colour a shape for your face or mask.
An **ellipse** is an oval shape. If you specify the same width and height, you will draw a circle.

Add code to the `draw()` function to set the fill colour using red, green and blue values, just like you did for the background.

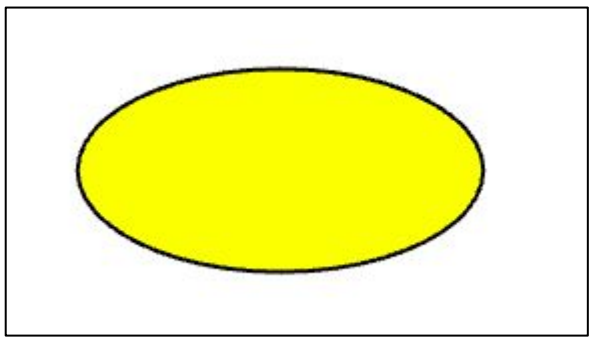
Then, draw a circle in this colour.

Test: Run your code and you should see a coloured circle.

```
def draw():
    # Put code to run every frame here
    background(255, 255, 255)
    # Add code to draw your face here
    fill(255, 255, 0)
    ellipse(
        screen_size/2,
        screen_size/2,
        200,
        200
    )
```

Change the width and height values to see the ellipse change shape.

```
def draw():
    # Put code to run every frame here
    background(255, 255, 255)
    # Add code to draw your face here
    fill(255, 255, 0)
    ellipse(
        screen_size/2,
        screen_size/2,
        100, # width
        50   # height
    )
```



Before the code where you draw the ellipse, you can choose to set a colour and thickness.

```
    fill(255, 255, 0)
    stroke(255, 255, 255)
    stroke_weight(3)
    ellipse(
        screen_size/2,
        screen_size/2,
        100,
        50
    )
```

Or, if you prefer, you can remove the stroke and have no outline.

```
14        fill(255, 255, 0)
15        no_stroke()
16        ellipse(
17            screen_size/2,
18            screen_size/2,
19            100,
20            50
21        )
```

Test: Experiment with changing the stroke colour and thickness or removing it, then run your code to see the results.

Rectangles

Rectangles are drawn in almost the same way as an ellipse.

Change the function **ellipse** to instead call the function **rect**.

```
12  def draw():
13      # Put code to run every frame here
14      background(255, 255, 255)
15      # Add code to draw your face here
16      fill(255, 255, 0)
17      rect(
18          screen_size/2,
19          screen_size/2,
20          100,
21          50
22      )
```

The **rect( )** function requires the x and y coordinates for the rectangle's top-left corner (or center, depending on the mode) and its width and height.

Test: Run your code to see a rectangle instead of an ellipse.

```
12  def draw():
13      # Put code to run every frame here
14      background(255, 255, 255)
15      # Add code to draw your face here
16      fill(255, 255, 0)
17      rect(
18          80, # x coordinate
19          60, # y coordinate
20          100,
21          50
22      )
```

The first two values for **rectangle** and **ellipse** represent the x, y coordinates of the centre of the shape. At the moment they are set to **screen_size/2** to position the shape in the centre of the screen.

The top left corner of the screen is coordinate **0,0**.

Increasing the **x** value will move the shape to the right. Increasing the **y** value will move the shape downwards. Change the position values to alter where the shape appears on the screen.

Test: Experiment with changing the coordinates, then run your code to see where the ellipse or rectangle is displayed.

Triangles

To draw a triangle, you need to give three pairs of values. Each pair is an x,y coordinate for one of the points of the triangle.

Add code to draw a **triangle**.

```
12  def draw():
13      # Put code to run every frame here
14      background(255, 255, 255)
15      # Add code to draw your face here
16      fill(255, 255, 0)
17      triangle(
18          210, 250,
19          330, 150,
20          220, 160
21      )
```

Test: Experiment with changing the coordinates, then run your code to see a triangle displayed by joining the dots at those coordinates.

Overlap shapes

A mouth is a great way of showing emotion. Will your character have a smile, frown, or something else?

You can create unique and complex designs by overlapping different shapes. for example you could add two overlapping circles to create a smile.

Start with an ellipse to represent the face.

```
def draw():
    # Put code to run every frame here
    background(255, 255, 255)
    # Add code to draw your face here
    no_stroke()
    fill(125, 75, 0) # Brown
    ellipse(200, 220, 150, 150) # Face
```

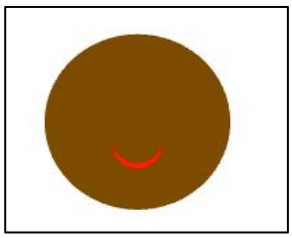
Set the **fill** colour for your mouth then draw an **ellipse**.

```
def draw():
    # Put code to run every frame here
    background(255, 255, 255)
    # Add code to draw your face here
    no_stroke()
    fill(125, 75, 0) # Brown
    ellipse(200, 220, 150, 150) # Face
    fill(255, 0, 0)  # Red
    ellipse(200, 240, 40, 40) # Mouth
```

Set the **fill** colour to match the face colour, then draw a second **ellipse**.
Change the **y** coordinate of the second **ellipse** to a slightly higher position for a smile.

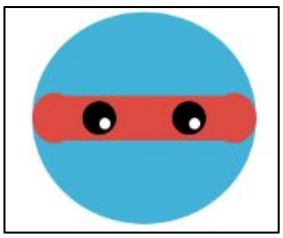
```
    fill(255, 0, 0)  # Red
    ellipse(200, 240, 40, 40) # Mouth
    fill(125, 75, 0) # Brown
    ellipse(200, 235, 40, 40) # Overlap
```

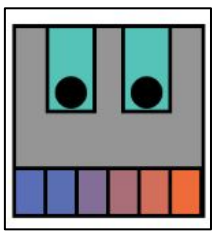
Test: Experiment with changing the fill colours and sizes of the ellipses. Run your program to see the results.

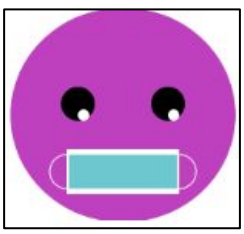


Create your face

Now it is over to you! Use the skills you have learned to draw shapes and create a face. Here are some examples for inspiration







This resource is licensed by the Raspberry Pi Foundation under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International licence. To view a copy of this license, visit, see creativecommons.org/licenses/by-nc-sa/4.0/.